Corporación Unificada Nacional de Educación Superior

VIGILADA MINEDUCACIÓN

Información general

Fecha de creación	6 – 10 - 2025	**Dirección de operaciones**

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Conexión y Consumo de APIs

Introducción

Imagina que estás en un restaurante concurrido. Tú (el cliente) tienes una necesidad clara: comer. La cocina tiene la capacidad de satisfacerla. Pero no entras directamente a la cocina a revolver las ollas y tomar los ingredientes. Sería un caos ineficiente y peligroso. En su lugar, interactúas con un intermediario experto: el mesero. Le entregas un pedido claro y estructurado (una **solicitud**) y él se encarga de traducirlo al lenguaje de la cocina, gestionar el proceso y, finalmente, traerte el plato terminado (una **respuesta**).

En el vasto y dinámico universo del software, una **API (Interfaz de Programación de Aplicaciones)** cumple exactamente ese rol de mesero. Es un contrato formal, un conjunto de reglas y herramientas que permite que dos piezas de software completamente distintas se comuniquen y colaboren entre sí. Una aplicación no necesita conocer la complejidad interna de otra —en qué lenguaje fue escrita, cómo está estructurada su base de datos o en qué servidor se ejecuta—; solo necesita saber "hablar" el idioma definido por la API.

Esta capacidad de interconexión estandarizada ha provocado una revolución silenciosa. Hemos pasado de construir aplicaciones como fortalezas aisladas (monolitos) a diseñar ecosistemas vibrantes y modulares (microservicios). Piensa en las aplicaciones que usas a diario en Colombia: cuando pides un servicio en Rappi, la app no ha construido su propio sistema de mapas, ni su propia pasarela de pagos, ni su propio sistema de notificaciones. En su lugar, consume la API de Google Maps para las rutas, la de Wompi o PayU para las transacciones y la de Firebase para los mensajes. Rappi es, en esencia, una brillante orquestación de múltiples APIs. Este modelo, conocido como la **"Economía de APIs"**, es el motor del desarrollo tecnológico actual.

Dominar la conexión y el consumo de APIs se ha convertido, por tanto, en una competencia esencial y diferenciadora para cualquier ingeniero de software. Las empresas ya no solo buscan programadores que puedan construir algoritmos, sino arquitectos de soluciones que puedan integrar servicios existentes para crear valor de forma rápida y escalable (Pressman & Maxim, 2020). Por ello, lo que aprenderás aquí va más allá de la sintaxis y los comandos. Aprender a consumir APIs no es solo una habilidad técnica; **es obtener la llave para conectar tus proyectos con el universo digital**.

Objetivos

Objetivo General

Al finalizar el estudio de este documento, el estudiante de la Corporación Unificada Nacional de Educación Superior estará en capacidad de diseñar, consumir y gestionar conexiones con APIs RESTful de forma segura y eficiente, integrándolas como componentes modulares dentro de aplicaciones orientadas a objetos construidas en Java, para construir soluciones de software robustas y escalables.

Objetivos Específicos

Para alcanzar este objetivo general, el estudiante deberá ser capaz de:

1. **Identificar los Fundamentos:** Describir y diferenciar los conceptos teóricos clave que sustentan la comunicación web, incluyendo los verbos y códigos de estado del protocolo HTTP, los principios de la arquitectura REST y la estructura del formato de datos JSON.
2. **Analizar en Contexto:** Evaluar el rol estratégico de las APIs en aplicaciones de la vida real y casos de estudio profesionales, explicando cómo su integración permite resolver problemas de negocio complejos y generar nuevas funcionalidades.
3. **Implementar la Comunicación:** Construir clientes HTTP en Java para ejecutar las cuatro operaciones básicas (CRUD) contra un endpoint de una API, enviando solicitudes y gestionando las respuestas del servidor de manera programática.
4. **Manejar los Datos:** Aplicar técnicas de serialización y deserialización para mapear eficientemente los datos entre el formato JSON de la API y los objetos del dominio de una aplicación Java, utilizando librerías estándar del mercado.
5. **Garantizar la Seguridad:** Comprender e implementar flujos de autenticación y autorización, como OAuth2, para consumir APIs protegidas y asegurar un manejo responsable de los datos y los permisos.

Fundamentos teóricos

Para construir un edificio sólido, primero debemos entender las propiedades de los materiales y los principios de la arquitectura. De igual manera, antes de consumir nuestra primera API, es crucial dominar los conceptos teóricos que gobiernan la comunicación entre aplicaciones en la web. Este capítulo establece esas bases fundamentales.

1.1 ¿Qué es una API? Conceptos Clave

Como vimos, una API es un intermediario, pero esa comunicación se rige por un vocabulario técnico preciso. Los componentes esenciales que debes dominar son:

- **Endpoint:** Es la URL específica donde la API expone un recurso. Piensa en ello como la "dirección" exacta a la que debes enviar tu solicitud. Por ejemplo, https://api.banco.com.co/v1/clientes/12345 es un endpoint que apunta al cliente con el ID 12345.
- **Recurso (Resource):** Es cualquier objeto o pieza de información que la API puede proporcionar. En el ejemplo anterior, el "cliente" es el recurso. Otros ejemplos podrían ser un producto, una publicación o un video.
- **Verbos HTTP (Métodos):** Son las acciones que deseamos realizar sobre un recurso. Los más comunes, que forman el acrónimo CRUD (Create, Read, Update, Delete), son:
    - **GET:** Para leer o recuperar un recurso. Es una operación segura, no modifica datos.
    - **POST:** Para crear un nuevo recurso. Por ejemplo, registrar un nuevo cliente.
    - **PUT:** Para actualizar completamente un recurso existente.
    - **DELETE:** Para eliminar un recurso.

- **Códigos de Estado HTTP:** Son números de tres dígitos que la API devuelve en su respuesta para indicar el resultado de la solicitud. Permiten una gestión de errores robusta. Se agrupan en familias (Comer, 2015):
    - **2xx (Éxito):** 200 OK, 201 Created.
    - **4xx (Error del Cliente):** 400 Bad Request, 401 Unauthorized, 404 Not Found.
    - **5xx (Error del Servidor):** 500 Internal Server Error.

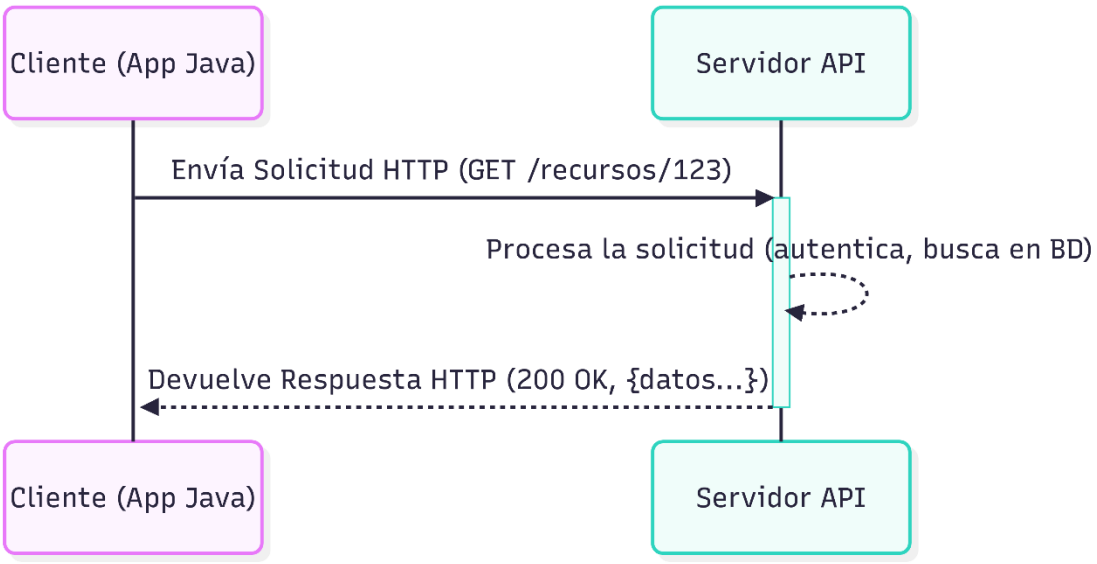

Figura 1: Diagrama del Ciclo de Vida de una Solicitud HTTP

Diagrama de secuencia que muestra a un Cliente enviando una solicitud HTTP a un Servidor API, el cual procesa la solicitud y devuelve una respuesta HTTP con un código de estado y un cuerpo de datos.

1.2 Arquitectura REST (Representational State Transfer)

No todas las APIs se diseñan de la misma manera. REST no es un protocolo ni un estándar, sino un conjunto de principios o restricciones arquitectónicas para diseñar aplicaciones en red. Las APIs que siguen estos principios se denominan **APIs RESTful** y son el estándar de facto en la web por su simplicidad y rendimiento (Pressman & Maxim, 2020).

Los principios clave son:

- **Arquitectura Cliente-Servidor:** Clara separación entre el cliente (quien consume los datos) y el servidor (quien los almacena y provee).
- **Sin Estado (Stateless):** Cada solicitud del cliente al servidor debe contener toda la información necesaria para ser entendida y procesada. El servidor no guarda ningún estado del cliente entre solicitudes.
- **Cacheable:** Las respuestas del servidor deben poder ser almacenadas en caché por el cliente para mejorar el rendimiento.
- **Interfaz Uniforme:** Es el principio fundamental de REST. Simplifica la arquitectura al requerir una forma estandarizada de comunicación, que

incluye la identificación de recursos vía URI (el endpoint), la manipulación de recursos a través de representaciones (como JSON) y el uso de mensajes autodescriptivos.

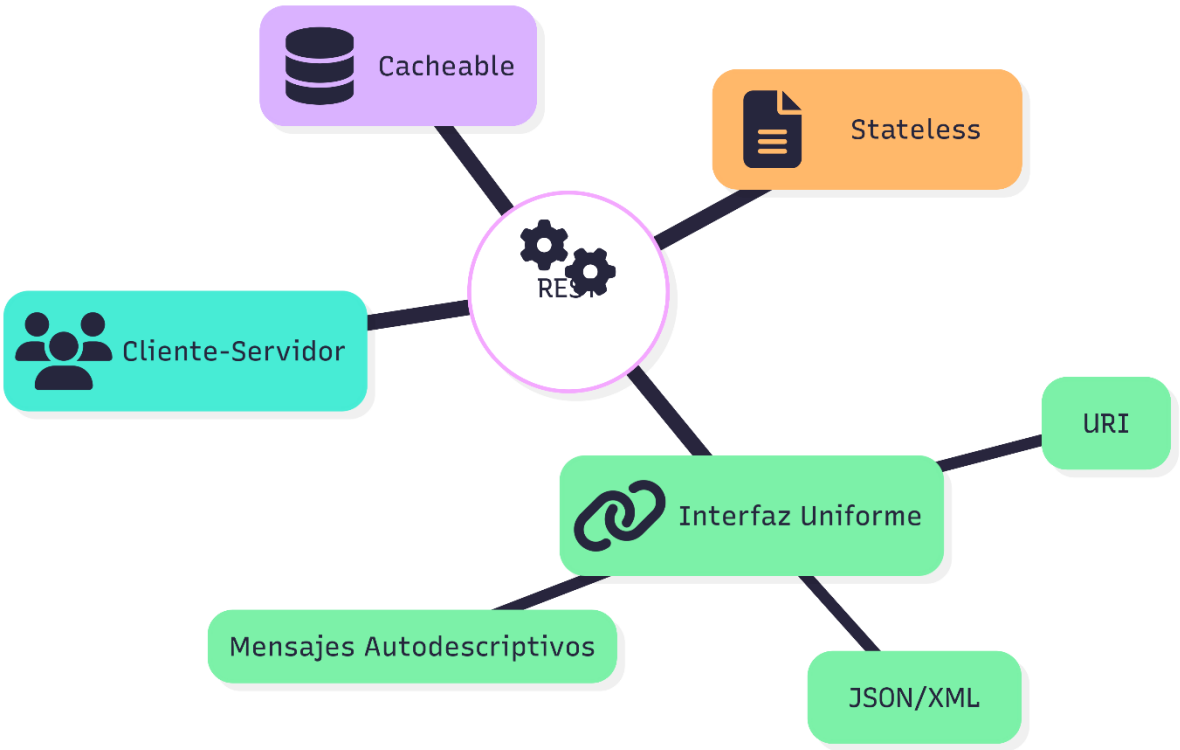

Figura 2: Mapa Conceptual de los Principios REST

Mapa conceptual que muestra el concepto central "REST" conectado a sus cuatro principios principales: Cliente-Servidor, Sin Estado, Cacheable e Interfaz Uniforme.

1.3 El Formato de Datos JSON

Para que dos sistemas intercambien datos, deben "hablar" un idioma común. **JSON (JavaScript Object Notation)** se ha convertido en el lenguaje preferido para las APIs RESTful debido a que es ligero, legible para los humanos y fácil de procesar para las máquinas. Su estructura se basa en pares de clave: valor, similar a los atributos de un objeto en programación, lo que facilita enormemente su mapeo (Yanev, 2025).

Un ejemplo de un objeto Estudiante en JSON:

JSON
```
{
  "id": "2025102001",
  "nombre": "Ana",
```

```
  "apellido": "Rojas",
  "activo": true,
  "cursos": [ "POO2", "Bases de Datos" ]
}
```

1.4 Solicitudes y Respuestas HTTP con Objetos

El verdadero poder de usar APIs en Programación Orientada a Objetos radica en la capacidad de convertir datos JSON en objetos de nuestro lenguaje (en este caso, Java) y viceversa.

- **Deserialización:** Es el proceso de tomar una cadena de texto en formato JSON recibida del servidor y convertirla en un objeto Java. Por ejemplo, el JSON anterior se convertiría en una instancia de una clase Estudiante en nuestro código.
- **Serialización:** Es el proceso inverso. Tomar un objeto Java (por ejemplo, uno que acabamos de crear) y convertirlo a una cadena de texto JSON para enviarlo al servidor en el cuerpo de una solicitud POST o PUT.

Este mapeo directo entre el formato de datos de la API y los objetos de nuestro programa es lo que permite una integración limpia y coherente, siguiendo los principios del encapsulamiento y la abstracción de la POO (Deitel & Deitel, 2018; Erl, 2008).

Aplicaciones en la vida cotidiana

La teoría del Capítulo 1 cobra vida cuando observamos el ecosistema digital que nos rodea. Las APIs no son un concepto abstracto confinado a los libros de texto; son la infraestructura que permite que las aplicaciones de nuestro celular, las plataformas de comercio electrónico y los servicios gubernamentales funcionen de manera integrada y eficiente. Este capítulo aterriza los fundamentos en casos de uso cotidianos, demostrando el valor práctico y transformador de saber consumir APIs.

2.1 APIs en las Redes Sociales: El Mundo Conectado

Cada vez que ves un "Iniciar sesión con Google" o "Continuar con Facebook" en una aplicación, estás presenciando una API de autenticación en acción. En lugar de crear un nuevo usuario y contraseña, la aplicación delega la tarea de verificar tu identidad a la API de un proveedor de confianza. De manera similar, cuando una aplicación de noticias muestra los últimos trinos sobre un tema o una tienda en línea te permite compartir un producto en tu muro de Facebook, no están "hackeando" la plataforma; están utilizando sus APIs públicas.

- **Caso Práctico:** Un portal de marketing digital en Colombia, como "Marketing 4 Ecommerce", podría desarrollar un *dashboard* para sus clientes. Este tablero, en lugar de obligar al usuario a entrar a cada red social por separado, consume la API de Twitter para mostrar métricas de trinos, la API de Instagram para analizar el rendimiento de las historias y la API de Facebook para rastrear el alcance de las publicaciones. El valor para el cliente es inmenso: una vista unificada y en tiempo real de toda su estrategia digital, posible gracias a la orquestación de múltiples APIs.

2.2 APIs en el E-commerce Colombiano: Facilitando las Transacciones

El comercio electrónico en Colombia ha crecido exponencialmente, y las APIs son su columna vertebral. Desde la validación de la dirección de envío hasta el procesamiento del pago, cada paso es gestionado por servicios especializados que se comunican vía API.

- **Caso Práctico:** Imagina una pequeña empresa de Medellín que vende café de especialidad a través de su sitio web.
    1. **Cálculo de Envío:** Cuando un cliente en Cali ingresa su dirección, el sitio web no tiene una base de datos con todos los costos de envío del país. En su lugar, envía los datos del paquete (peso, dimensiones) y la dirección de destino a la **API de un operador logístico** como Servientrega o Interrapidísimo. La API calcula el costo en tiempo real y lo devuelve para mostrarlo en el carrito de compras.

2. **Procesamiento del Pago:** Al momento de pagar, el cliente elige su método preferido (tarjeta de crédito, PSE, etc.). El sitio web no procesa directamente esta información sensible. Envía los detalles de la transacción a la **API de una pasarela de pagos** como Wompi (de Bancolombia) o PayU. Esta API se encarga de la comunicación segura con el banco, autoriza la transacción y devuelve una confirmación (o un rechazo) a la tienda.

Este desacoplamiento es crucial: la tienda de café se enfoca en lo que mejor sabe hacer (vender café), mientras que la logística y los pagos son manejados por expertos a través de APIs robustas y seguras. La integración de APIs de inteligencia artificial, por ejemplo, puede incluso personalizar la experiencia de compra, ofreciendo productos basados en el historial del cliente (Yanev, 2025).

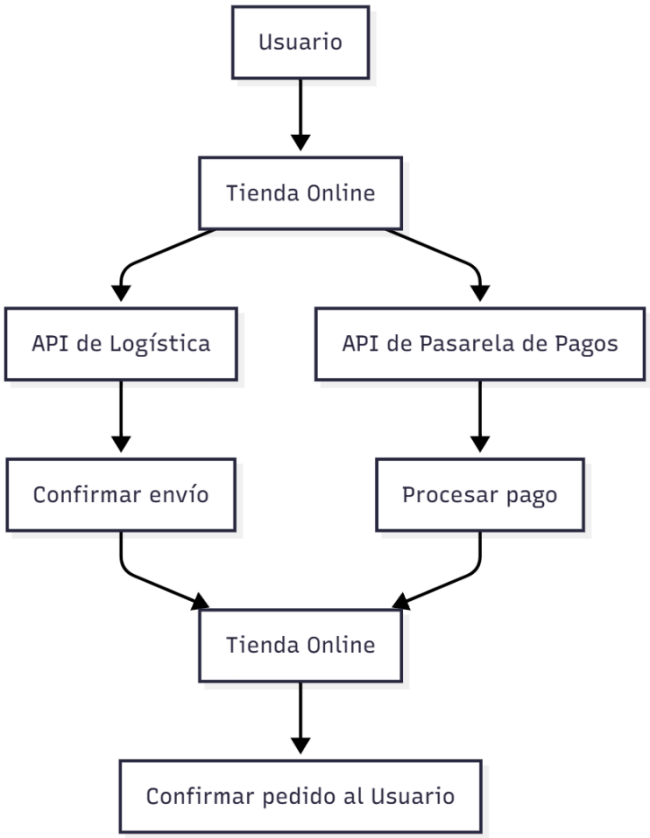

Figura 3: Diagrama de Flujo de una Compra Online con APIs

Diagrama de flujo que ilustra los pasos de una compra en línea. El flujo comienza con el Usuario, interactúa con la Tienda Online, que a su vez se comunica con una API de Logística para el envío y una API de Pasarela de Pagos para la transacción, antes de confirmar el pedido al Usuario.

2.3 APIs de Servicios Públicos y Datos Abiertos: Innovación Cívica

Los gobiernos y las entidades públicas también han adoptado las APIs como un mecanismo para fomentar la transparencia y la innovación. A través de portales de datos abiertos, exponen información de interés público para que desarrolladores, periodistas y ciudadanos puedan crear nuevas aplicaciones y servicios de valor.

- **Ejemplo Cercano:** El portal de **Datos Abiertos de Colombia (Datos.gov.co)** ofrece cientos de conjuntos de datos a través de APIs. Un estudiante de la CUN podría desarrollar una aplicación que:
    - Consuma la API de **Calidad del Aire** del Ministerio de Ambiente para mostrar un mapa en tiempo real de los niveles de contaminación en las principales ciudades.
    - Utilice la API del **SECOP (Sistema Electrónico de Contratación Pública)** para crear un sistema de alertas que notifique a las pequeñas empresas sobre nuevas licitaciones en su sector.
    - Integre la API de **Indicadores del Banco de la República** para crear una herramienta de finanzas personales que convierta precios a la TRM del día.

Estas aplicaciones no solo representan excelentes proyectos académicos, sino que también son ejemplos de cómo la tecnología, habilitada por las APIs, puede generar un impacto social positivo, mejorar la toma de decisiones y fortalecer la participación ciudadana. La correcta evaluación y testeo de estas APIs es fundamental para asegurar su fiabilidad y seguridad (Qin et al., 2025).

Casos de estudio

Mientras que el capítulo anterior mostró el "qué" y el "dónde" de las APIs, este se enfoca en el "cómo" y el "porqué" dentro de un contexto empresarial. Analizaremos un desafío de negocio específico, desglosaremos la solución técnica implementada y evaluaremos los resultados cuantitativos y cualitativos. Este enfoque integral es el que se utiliza en la industria para justificar y desarrollar proyectos de software.

3.1 Caso de Estudio: "CunExpress - Logística Inteligente"

Contexto: CunExpress es una empresa de mensajería y paquetería pyme con sede en Bogotá. Su operación se ha basado tradicionalmente en la asignación manual de rutas, donde un despachador, basándose en su experiencia, distribuye los paquetes del día entre los mensajeros.

- **Desafío:** La empresa enfrenta crecientes problemas de competitividad. Sus costos de combustible son altos, las entregas a menudo se retrasan debido al tráfico impredecible de Bogotá y a eventos inesperados (como manifestaciones o fuertes lluvias), y la satisfacción del cliente ha comenzado a disminuir. La gerencia necesita una solución tecnológica para optimizar la operación logística sin incurrir en el costo prohibitivo de desarrollar un sistema de enrutamiento desde cero.
- **Solución Aplicada:** Un equipo de tecnólogos de la CUN es contratado para desarrollar un módulo de software en Java que se integre al sistema de gestión de pedidos existente. La solución no busca reinventar la rueda, sino conectar inteligentemente servicios externos a través de sus APIs. La arquitectura propuesta se basa en el consumo de dos APIs clave:

    1. **API de Google Maps (Directions API):** Para el cálculo de rutas. Al inicio del día, el módulo de Java toma la lista de direcciones de entrega, la envía a la API de Google Maps y esta devuelve la ruta más eficiente que conecta todos los puntos, considerando el tráfico en tiempo real.
    2. **API de OpenWeatherMap (One Call API):** Para la predicción climática. Antes de despachar a un mensajero, el sistema consulta las coordenadas de la ruta en la API de OpenWeatherMap. Si se pronostica una lluvia intensa en una zona específica, el sistema puede recalcular la ruta para evitar áreas propensas a inundaciones o simplemente notificar al cliente de un posible retraso.

El núcleo de la solución es un conjunto de clases Java (GestorRutas, ClienteGoogleMaps, ClienteClima) que encapsulan la lógica de las llamadas a las APIs. Por ejemplo, el método obtenerRutaOptima() de GestorRutas se encarga de recolectar las direcciones, construir la solicitud para la API de Google Maps,

enviarla, y deserializar la respuesta JSON para convertirla en un objeto Ruta que el sistema puede entender y mostrar.

- **Impacto y Datos:** Tras un piloto de tres meses, CunExpress midió los siguientes resultados:
    - **Reducción del 15% en el tiempo promedio de entrega:** Gracias a la optimización de rutas y la evasión de trancones.
    - **Disminución del 10% en el consumo de combustible:** Rutas más cortas y eficientes se traducen directamente en ahorro de costos.
    - **Aumento del 20% en la satisfacción del cliente:** Medido a través de encuestas, gracias a entregas más puntuales y notificaciones proactivas sobre retrasos por clima.

Este caso demuestra cómo la integración de APIs es una estrategia de **apalancamiento tecnológico**. CunExpress, una pyme, logró acceder a la misma tecnología de optimización de rutas que usan gigantes como Amazon o Mercado Libre, pagando solo por el consumo de las APIs, lo cual es significativamente más económico y rápido que el desarrollo propio (Pressman & Maxim, 2020).

Característica	Antes de la Integración de APIs	Después de la Integración de APIs
Asignación de Ruta	Manual, basada en la experiencia del despachador.	Automática, optimizada con datos de tráfico en tiempo real.
Adaptabilidad	Nula. Las rutas no cambiaban ante imprevistos.	Alta. Capacidad de recalcular rutas por clima o tráfico.
Toma de Decisiones	Subjetiva y propensa a errores humanos.	Basada en datos, objetiva y consistente.
Comunicación Cliente	Reactiva. El cliente llama para saber de su paquete.	Proactiva. Notificaciones automáticas de posibles retrasos.
Costo Operativo	Alto (más combustible, más horas-hombre).	Reducido (ahorro en combustible y tiempo).

Tabla 1: Comparativa de Funcionalidades (Antes vs. Después de la API)

Tabla comparativa que muestra las mejoras en el proceso de asignación de rutas de una empresa de logística antes y después de integrar APIs, destacando cambios en eficiencia, adaptabilidad y comunicación.

Este enfoque, que consiste en componer sistemas complejos a partir de servicios más pequeños y especializados, es la esencia de las arquitecturas orientadas a servicios (SOA) y microservicios, un paradigma fundamental en el desarrollo de

software moderno (Erl, 2008). La habilidad para identificar, evaluar e integrar APIs de terceros es, por lo tanto, una competencia estratégica para cualquier profesional del software.

Herramientas y técnicas

La transición de la teoría a la práctica requiere el dominio de herramientas específicas. En el ecosistema de Java, existen librerías robustas y estandarizadas para manejar la comunicación por red y el procesamiento de datos. Este capítulo se enfoca en el "cómo": cómo escribir el código para realizar solicitudes HTTP, cómo manipular datos en formato JSON y cómo implementar flujos de autenticación para construir las soluciones que describimos en capítulos anteriores.

4.1 Cliente HTTP Nativo en Java

Desde la versión 11, Java incluye un cliente HTTP moderno y muy capaz en su biblioteca estándar (java.net.http), eliminando la necesidad de dependencias externas para tareas básicas. Este cliente es asíncrono por defecto y tiene una API fluida y fácil de usar.
Los tres componentes principales son:

- HttpClient: El objeto que envía las solicitudes y recibe las respuestas. Se crea una sola vez y se reutiliza.
- HttpRequest: Representa la solicitud que enviaremos. Se construye especificando la URI (el endpoint), el método (GET, POST, etc.) y las cabeceras.
- HttpResponse: Contiene la respuesta del servidor, incluyendo el código de estado, las cabeceras y el cuerpo (los datos).
-

Ejercicio Práctico: Realizar una solicitud GET
El siguiente código consume la API pública "JSONPlaceholder" para obtener los datos de un usuario de prueba.

Java
```
// Es necesario importar las clases del paquete java.net.http
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ConsumoAPI {

    public static void main(String[] args) throws Exception {
        // 1. Crear una instancia de HttpClient
        HttpClient client = HttpClient.newHttpClient();

        // 2. Crear una instancia de HttpRequest con la URI del endpoint
        HttpRequest request = HttpRequest.newBuilder()
```

```
            .uri(URI.create("https://jsonplaceholder.typicode.com/users/1"))
            .GET() // Especificar el verbo HTTP
            .build();

        // 3. Enviar la solicitud y recibir la respuesta
        // El segundo argumento, BodyHandlers.ofString(), indica que queremos leer el
cuerpo como un String
        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

        // 4. Imprimir los resultados
        System.out.println("Código de Estado: " + response.statusCode());
        System.out.println("Cuerpo de la Respuesta (JSON): " + response.body());
    }
}
```

4.2 Manejo de JSON en Java con la Librería Gson

Como vimos, la respuesta de la API es una cadena de texto en formato JSON. Trabajar directamente con este texto es engorroso y propenso a errores. Aquí es donde entran en juego las librerías de mapeo objeto-JSON, como **Gson** de Google o **Jackson**. Estas herramientas automatizan la **serialización** (de objeto Java a JSON) y la **deserialización** (de JSON a objeto Java).
Para usar Gson, primero debes agregar la librería a tu proyecto en NetBeans. Luego, creas una clase en Java (un "POJO" - Plain Old Java Object) que tenga los mismos nombres de atributos que las claves del JSON.

Ejercicio Práctico: Deserializar la respuesta JSON a un Objeto

1. **Crear la clase Usuario.java:**

Java

```
public class Usuario {
    private int id;
    private String name;
    private String username;
    private String email;

    // Es importante generar los getters y un método toString() para ver el resultado
    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
```

```
    @Override
    public String toString() {
        return "Usuario{id=" + id + ", name='" + name + "', email='" + email + "'}";
    }
}
```

2. **Modificar el main para usar Gson:**

Java

```
import com.google.gson.Gson; // Importar Gson
// ... (otras importaciones)

public class ConsumoAPI {
    public static void main(String[] args) throws Exception {
        // ... (código para la solicitud HTTP)
        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

        // 5. Usar Gson para deserializar el JSON
        if (response.statusCode() == 200) {
            Gson gson = new Gson();
            // response.body() contiene el String JSON
            Usuario usuario = gson.fromJson(response.body(), Usuario.class);

            System.out.println("Objeto Usuario deserializado: " + usuario);
            System.out.println("Nombre del usuario: " + usuario.getName());
        } else {
            System.out.println("Error al consultar la API.");
        }
    }
}
```

Este enfoque es la base de una programación limpia y orientada a objetos, pues nos permite trabajar con Usuario como un objeto de nuestro dominio, en lugar de manipular cadenas de texto (Deitel & Deitel, 2018).

4.3 Autenticación y Autorización con OAuth2

Muchas APIs no son públicas y requieren que la aplicación se identifique y tenga permisos para acceder a los datos. **OAuth2** es un estándar de la industria para la autorización delegada. En lugar de que el usuario le dé su contraseña a nuestra aplicación, le da un permiso (consentimiento) a través de un intermediario de confianza (como Google o Facebook).

El flujo más común ("Authorization Code") funciona así, conceptualmente:

1. Nuestra aplicación redirige al usuario a la página de inicio de sesión del proveedor (ej. Google).

2. El usuario inicia sesión y autoriza a nuestra aplicación a acceder a ciertos datos (ej. "ver tu correo electrónico").
3. El proveedor redirige de vuelta al usuario a nuestra aplicación, entregándole un "código de autorización" de un solo uso.
4. Nuestra aplicación, desde el servidor (backend), intercambia este código (junto con sus propias credenciales de cliente) por un **Token de Acceso**.
5. Este Token de Acceso es el que se incluye en las cabeceras de las futuras solicitudes a la API para autenticar la comunicación.

La implementación completa de este flujo es avanzada, pero como tecnólogo, es crucial entender el proceso para poder utilizar librerías que lo simplifiquen y para configurar correctamente las credenciales de cliente en las consolas de desarrollador de las APIs (Pressman & Maxim, 2020).

4.4 Taller de Integración: NetBeans y Base de Datos SQL

Unamos todo en un taller práctico. El objetivo es crear una aplicación simple en NetBeans que consuma una API pública, procese los datos y los guarde en una tabla de una base de datos local (ej. MySQL o PostgreSQL).

Pasos del Taller:

1. **Configurar el Proyecto:** Crea un nuevo proyecto Java en NetBeans. Agrega la librería de Gson y el conector JDBC para tu base de datos al *classpath* del proyecto.
2. **Diseñar la Base de Datos:** Crea una tabla simple en tu gestor de base de datos. Por ejemplo: CREATE TABLE posts (id INT PRIMARY KEY, userId INT, title VARCHAR(255), body TEXT);.
3. **Consumir la API:** Usando HttpClient, haz una solicitud GET a https://jsonplaceholder.typicode.com/posts. Este endpoint devuelve un array de 100 publicaciones.
4. **Deserializar el Array:** La respuesta será un JSON que empieza con [. Debes usar Gson para deserializar esta cadena en un array o una lista de objetos Post (Post[] o List<Post>). Para ello, primero debes crear la clase Post.java.
5. **Conectar y Guardar:** Utiliza JDBC para establecer una conexión con tu base de datos. Itera sobre la lista de objetos Post y, para cada uno, ejecuta una sentencia INSERT en la tabla posts. Es fundamental usar PreparedStatement para evitar inyección SQL (Yanev, 2025).
6. **Verificar:** Ejecuta una consulta SELECT * FROM posts en tu base de datos para confirmar que los 100 registros se guardaron correctamente.

Este ejercicio integra redes, procesamiento de datos y persistencia, simulando un ciclo de trabajo muy común en el desarrollo de software profesional.

Conclusión

Hemos viajado desde los fundamentos teóricos que definen a una API como el "mesero" del mundo digital, hasta su aplicación práctica en escenarios cotidianos y profesionales que impactan la economía y la sociedad en Colombia. Desglosamos la arquitectura REST, manipulamos el formato JSON y analizamos cómo gigantes del e-commerce y servicios públicos dependen de estas interfaces para funcionar. Finalmente, nos sumergimos en el código, utilizando Java y herramientas profesionales para construir un puente tangible entre nuestra aplicación y el vasto universo de servicios disponibles en la web.

La lección más importante de este recorrido es que el desarrollo de software moderno es un acto de **integración** y **colaboración**. La habilidad para consumir APIs ya no es una especialización, sino una competencia fundamental. Te permite construir aplicaciones más potentes, en menos tiempo y con menos recursos, al apalancarte en el trabajo de miles de desarrolladores expertos en sus respectivos dominios, ya sea en mapas, pagos, inteligencia artificial o análisis de datos. Has aprendido a no reinventar la rueda, sino a construir vehículos más rápidos y eficientes conectando las mejores ruedas disponibles.

El conocimiento adquirido aquí es una base sólida, pero el ecosistema de APIs evoluciona a una velocidad vertiginosa. Te invito a ser un explorador constante: busca nuevas APIs, lee su documentacion, experimenta con ellas en pequeños proyectos personales y mantén viva tu curiosidad. La verdadera maestría no reside en memorizar una librería o un framework, sino en desarrollar la capacidad de entender, adaptar e integrar cualquier servicio que el futuro ponga a tu disposición.

Para finalizar, te dejo dos preguntas abiertas que te invitan a llevar esta reflexión más allá de lo técnico:

1. Más allá de la eficiencia técnica, ¿cuál crees que es la **responsabilidad ética** de un desarrollador al consumir APIs que manejan datos personales de los usuarios? ¿Dónde trazarías la línea entre la personalización de un servicio y la invasión de la privacidad?
2. Observando tu entorno local (tu barrio, tu ciudad, tu universidad), ¿qué **proceso manual o ineficiente** podrías mejorar o automatizar drásticamente mediante la integración de una o varias APIs que ya existen?

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

Referencias.

Comer, D. E. (2015). *Redes de computadoras e Internet*. Pearson Educación. https://www-ebooks7-24-com.zproxy.cun.edu.co/?il=3262

Deitel, P. J., & Deitel, H. M. (2018). *Cómo programar en Java* (11ª ed.). Pearson Educación. https://ebooks7-24-com.zproxy.cun.edu.co/?il=7333

Erl, T. (2008). *SOA: Principios de los servicios de diseño*. Pearson Educación. https://www-ebooks7-24-com.zproxy.cun.edu.co/?il=3351

Gómez, M. (2007). *Calidad en el desarrollo de software*. RA-MA. https://www-digitaliapublishing-com.zproxy.cun.edu.co/a/17094

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería de software: Un enfoque práctico* (9ª ed.). McGraw Hill Interamericana. https://ebooks7-24-com.zproxy.cun.edu.co/?il=24119

VV. AA. (2025). *Inteligencia Artificial. El futuro de la humanidad*. Aranzadi. https://www-digitaliapublishing-com.zproxy.cun.edu.co/a/188182